ELEC205 Assignment 3

Baran Ecertemel 0087279

Task a)

```
// Inputs: X (4-bit), Y (4-bit), Op (2-bit), Start (1-bit), Clock
// Outputs: Display_Result (7-segment), Carry_Out (1-bit), Overflow (1-bit)
// Internal Registers: Reg_X (4-bit), Reg_Y (4-bit), Reg_Op (2-bit), Temp_Result (4-bit)

STATE IDLE:
    Display_Result <- Old_Value or Disabled // Do not show new result until computation is finalized
    IF Start == 1 THEN
        // Latch inputs safely
        Reg_X <- X
        Reg_Y <- Y
        Reg_Op <- Op
        NEXT_STATE <- COMPUTE
    ELSE
        NEXT_STATE <- IDLE
    ENDIF

STATE COMPUTE:
    // Perform the selected operation
    SWITCH (Reg_Op)
        CASE 00: // Addition (X + Y)
            Temp_Result <- Reg_X + Reg_Y
            Carry_Out   <- Bit_4_of_Addition
            Overflow    <- (Reg_X[3] == Reg_Y[3]) AND (Temp_Result[3] != Reg_X[3])

        CASE 01: // Subtraction (X - Y)
            Temp_Result <- Reg_X - Reg_Y
            Carry_Out   <- Bit_4_of_Subtraction
            Overflow    <- (Reg_X[3] != Reg_Y[3]) AND (Temp_Result[3] != Reg_X[3])

        CASE 10: // Computation-1 (2X + Y/2)
            // Arithmetic shift left for 2X, arithmetic shift right for Y/2
            Temp_Result <- (Reg_X << 1) + (Reg_Y >>> 1)
            Carry_Out   <- 0 // Carry and overflow are not specified for this operation
            Overflow    <- 0

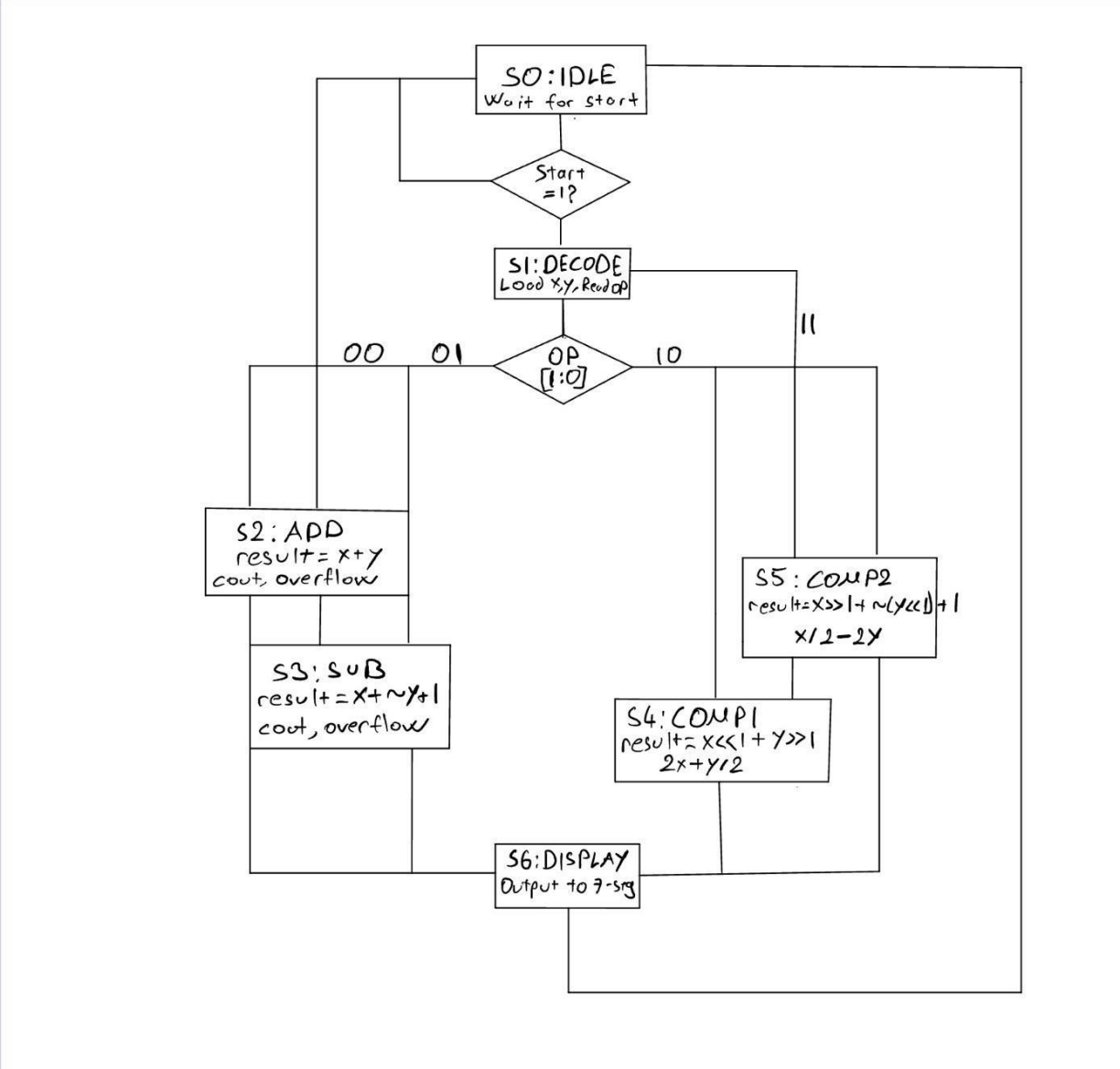
        CASE 11: // Computation-2 (X/2 - 2Y)
            // Arithmetic shift right for X/2, arithmetic shift left for 2Y
            Temp_Result <- (Reg_X >>> 1) - (Reg_Y << 1)
            Carry_Out   <- 0
            Overflow    <- 0
    ENDSWITCH

    NEXT_STATE <- DONE

STATE DONE:
    // Computation finalized, output result to 7-segment display
    Display_Result <- Temp_Result

    // Operation complete, return to IDLE state to wait for a new command
    NEXT_STATE <- IDLE
```

Task b)



Task c)

Input registers:

-REG_X 4-bit D flip-flop register, holds operand X

-REG_Y 4-bit D flip-flop register, holds operand Y

-REG_OP 2-bit register, holds the operation type

-REG_RESULT 4-bit register, holds the computed result

Functional units:

-SHL (Shift Left Logic) 4-bit, shifts X or Y one position left (×2)

-SHR (Shift Right Arithmetic) 4-bit, shifts X or Y one position right (÷2), MSB is replicated to preserve sign

-ADDER 4-bit ripple-carry adder, takes two operands, produces sum[3:0], Cout, OVF

Multiplexers:

-MUX_A 4:1 MUX, selects the A input to the adder (X, X<<1, X>>1)

-MUX_B 4:1 MUX, selects the B input to the adder (Y, ~Y+1, Y>>1, ~(Y<<1)+1)

Output unit:

SEG_DEC 4-bit to 7-segment decoder

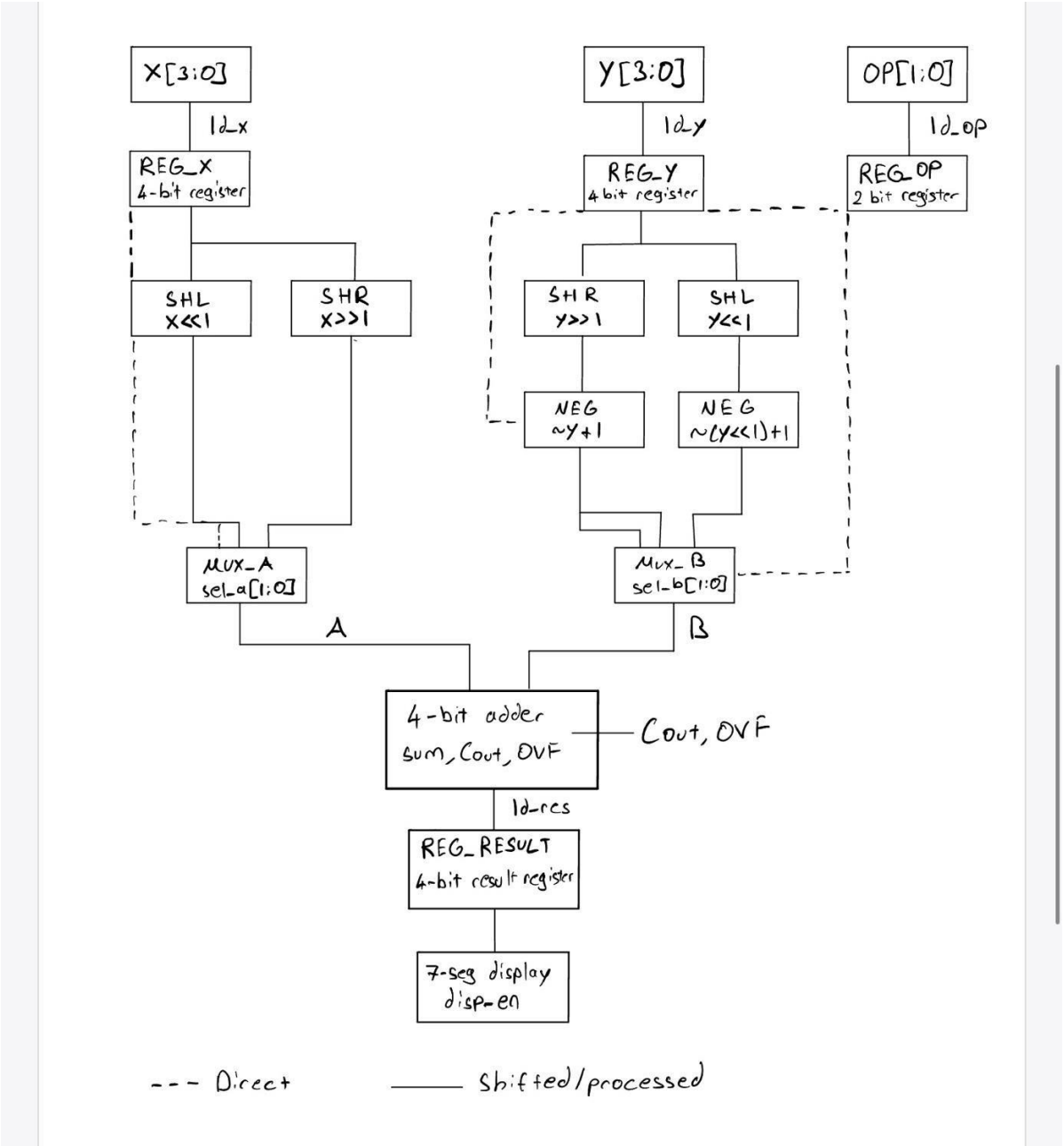
DISPLAY 7-segment display output

Control signals:

Signal	**Width**	**Description**
ld_x	1-bit	Load REG_X from input
ld_y	1-bit	Load REG_Y from input
ld_op	1-bit	Load REG_OP from input
Ld_res	1-bit	Latch adder output into REG_RESULT
sel_a[1:0]	2-bit	MUX_A selection signal
sel_b[1:0]	2-bit	MUX_B selection signal
disp_en	1-bit	Enable the 7-segment display

MUX selection table:

sel_a	A	sel_b	B input
00	X (raw)	00	Y (raw)
01	X << 1	01	~Y+1
10	X >> 1	10	Y >> 1
11	-	11	~(Y << 1) + 1



Task d)

The Control Unit is a Moore-type FSM encoded with 3 D flip-flops, yielding 7 states (S0–S6).

State Encoding

State	Q2Q1Q0	Description
S0	000	IDLE
S1	001	DECODE
S2	010	ADD
S3	011	SUB
S4	100	COMP1
S5	101	COMP2
S6	110	DISPLAY

Next-State Functions:

S0 → S1 : START = 1

S0 → S0 : START = 0

S1 → S2 : OP = 00

S1 → S3 : OP = 01

S1 → S4 : OP = 10

S1 → S5 : OP = 11

S2, S3, S4, S5 → S6 : (unconditional)

S6 → S : (unconditional)

Boolean equations for D flip-flop inputs:

S0 = Q2'.Q1'.Q0'

S1 = Q2'.Q1'.Q0

D2 = (S1.OP[1]) + (Q2⊕Q1)

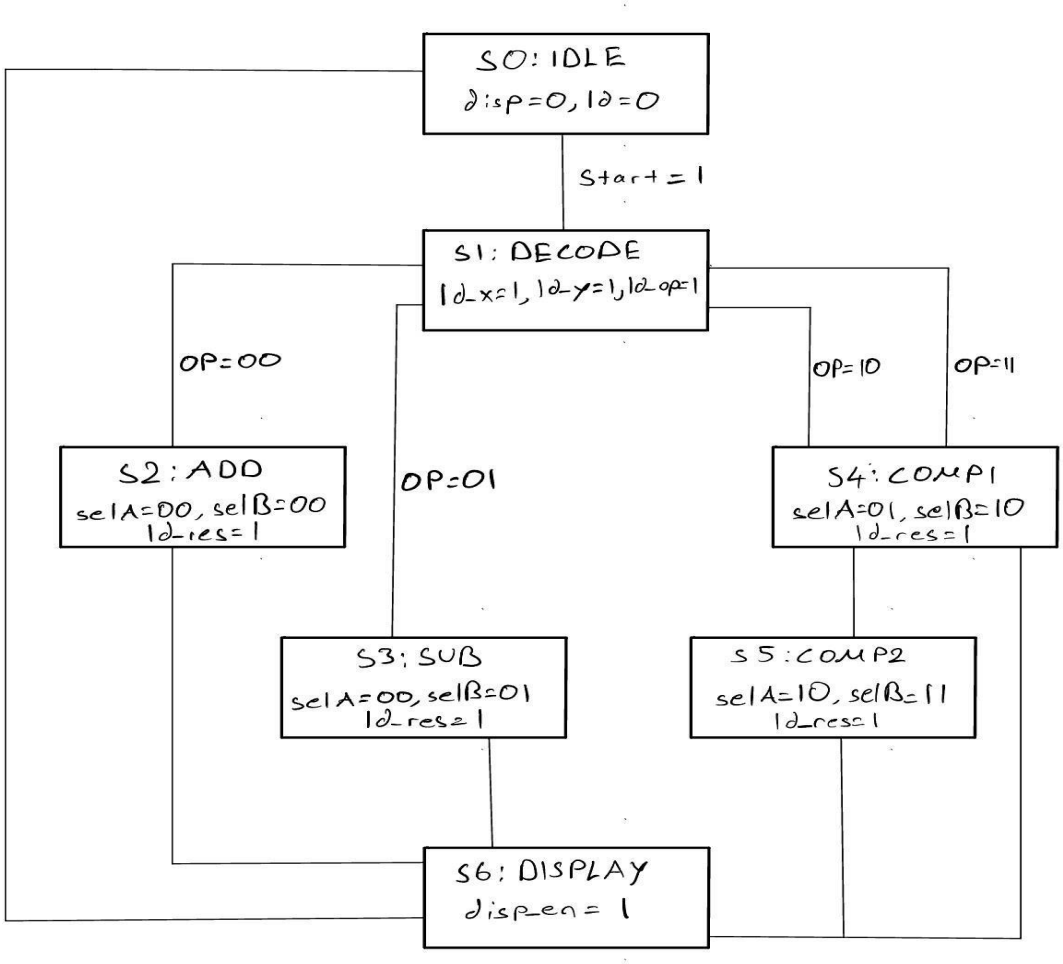
D1 = (S1.OP[1]') + (Q2⊕Q1)

D0 = (S0.START) + (S1.OP[0])

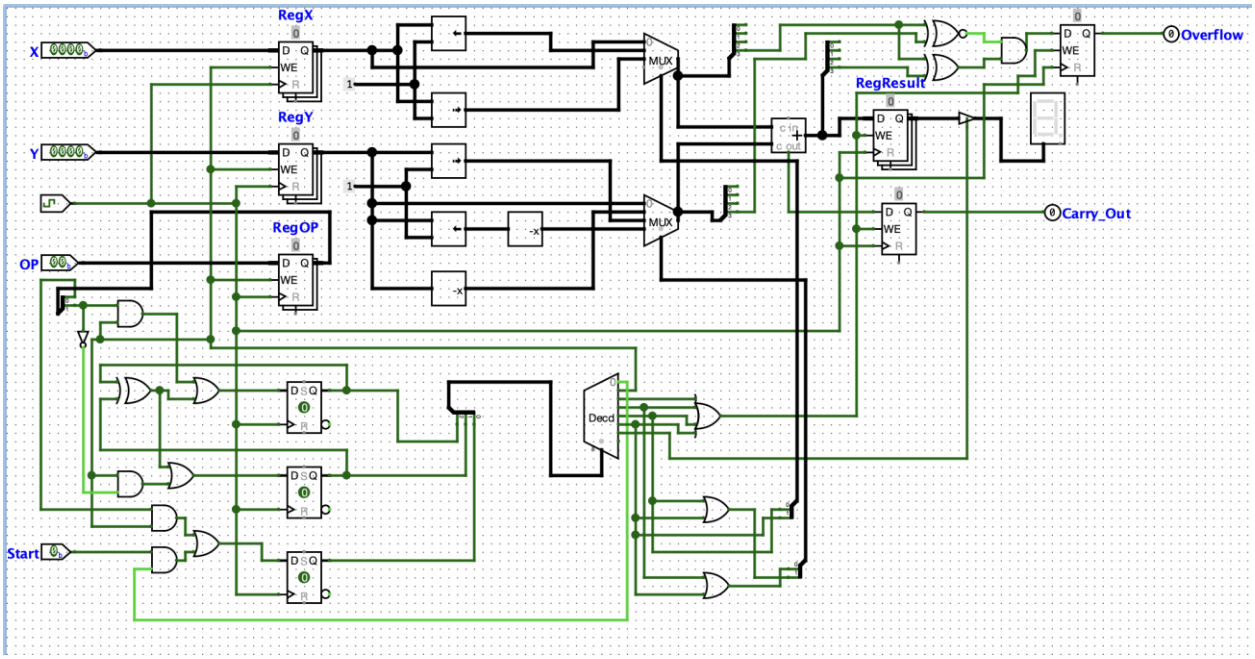
Output Functions (Moore-type)

Since the FSM is Moore-type, all outputs depend only on the current state:

State	ld_x	ld_y	ld_op	sel_a	sel_b	ld_res	disp_en
S0 IDLE	0	0	0	XX	XX	0	0
S1 DECODE	1	1	1	XX	XX	0	0
S2 ADD	0	0	0	00	00	1	0
S3 SUB	0	0	0	00	01	1	0
S4 COMP1	0	0	0	01	10	1	0
S5 COMP2	0	0	0	10	11	1	0
S6 DISPLAY	0	0	0	XX	XX	0	1



Task e)



The complete schematic of the ASM-based calculator designed in Logisim is presented in the figure above. The design strictly adheres to the proposed ASM chart and datapath architecture. It is partitioned into a Control Unit utilizing three D flip-flops and a 3-to-8 decoder for state transitions, and a Datapath utilizing a single shared 4-bit adder, multiplexers, shifters, and negators to perform the required arithmetic operations efficiently.

Task f)

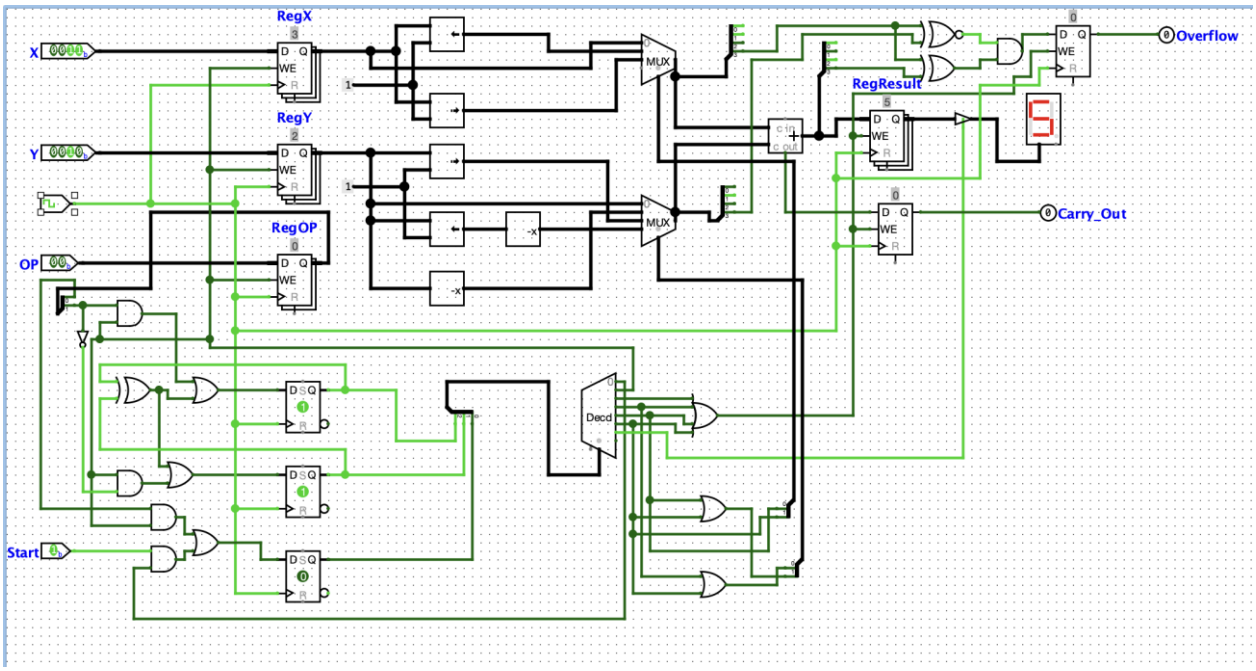
To verify the functionality of the implemented ASM design, various test cases were simulated in Logisim using the Poke Tool to manipulate inputs and the Clock ticks to observe state transitions. The results are displayed on the 7-segment display only during the S6 (DISPLAY) state, satisfying the design specifications.

Test Case 1: Addition (X + Y)

-Inputs: X = 3 (0011), Y = 2 (0010), OP = 00 (Addition)

-Expected Output: 3 + 2 = 5

-Observation: Upon activating the Start signal, the FSM transitioned through the states S0 -> S1 -> S2 -> S6. At state S6, the disp_en signal was triggered, and the Hex display correctly showed the result **5**.



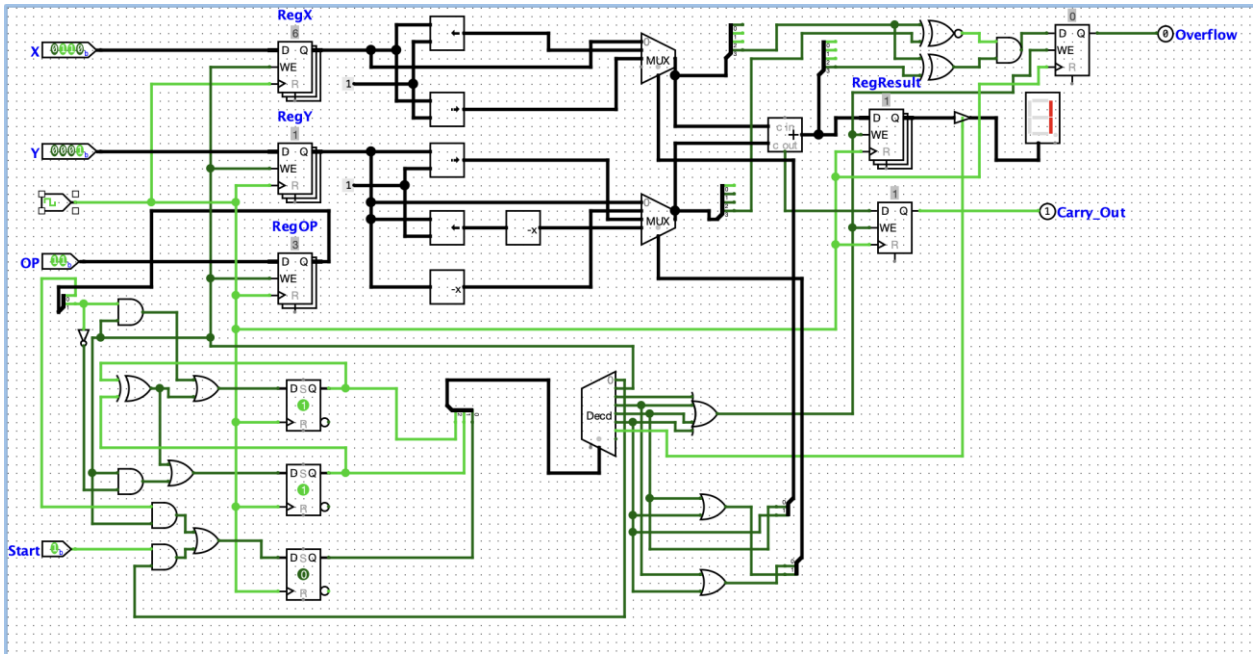
Test Case 1

Test Case 2: Computation-2 (X/2 – 2Y)

-Inputs: X = 6 (0110),Y = 1 (0001), OP = 11 (Computation-2)

-Expected Output: (6/2) – (2x1) = 3 – 2 = 1

-Observation: After setting the inputs and triggering Start, the control unit successfully navigated the path S0 -> S1 -> S5 -> S6. The datapath correctly routed the shifted values via the multiplexers to the shared adder. The final computed result displayed on the 7-segment screen is **1**.



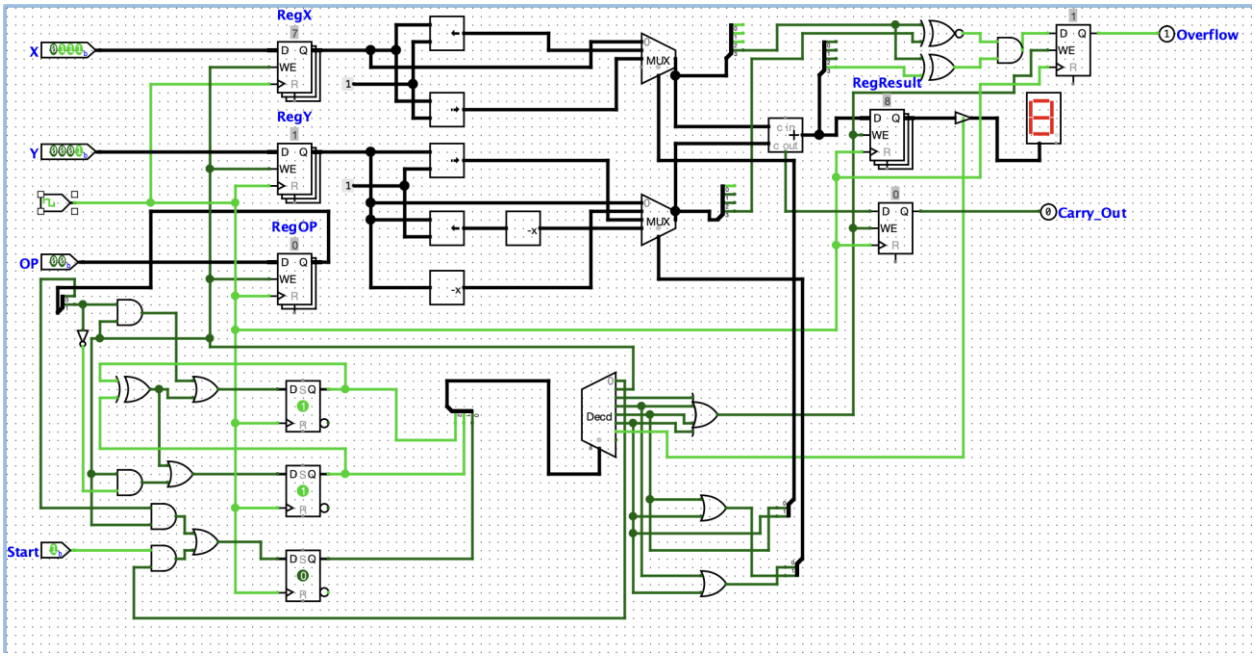
Test Case 2

Test Case 3: Addition with Overflow Detection

-Inputs: X = 7 (0111), Y = 1 (0001), OP = 00 (Addition)

-Expected Output: The mathematical sum of 7 + 1 is 8. However, in a 4-bit two's complement system, the maximum representable positive value is 7. Therefore, this operation exceeds the capacity, resulting in the binary value 1000 (which is -8 in two's complement and displayed as '8' on the Hex screen). The system must detect this anomaly and set the Overflow flag to '1'.

-Observation: Upon activating the Start signal, the FSM advanced to the S6 state. The Hex display showed '8'. Most importantly, the 1-bit Overflow output pin successfully lit up (became '1'), while the Carry_Out pin remained '0'. This perfectly verifies that the XNOR/XOR sign-bit comparison logic in the datapath correctly detects arithmetic overflow when two positive numbers produce a negative result.



Test Case 3